

Digi-Battle: An Augmented Reality Card Game Prototype

Abd El Rahaman Shehata
Dept. of Mobile Computing
University of Applied Sciences Upper Austria
Hagenberg, Austria
s2310455006@fhooe.at

Abstract—This paper presents Digi-Battle an Augmented Reality (AR) prototype developed with the Unity Game-Engine and Vuforia. The project aims to modernize the traditional Digimon trading card game by projecting interactive 3D models onto physical cards using mobile devices. This document details the architectural design, implementation of the AR tracking system, and the game logic used to create a turn-based battle experience.

I. INTRODUCTION

The main objective of this project is to create a prototype based on the popular anime franchise, Digimon. While the series originally features a physical trading card game, the experience can be significantly enhanced by bridging the physical and digital worlds using Augmented Reality (AR).

The prototype is structured into three distinct stages, each employing specific interaction mechanisms. The project leverages Unity Vuforia to detect physical cards for tracking purposes [1]. Currently, the system supports three specific cards corresponding to three different Digimon models.

II. DESIGN

As this project enhances an existing physical game, users are required to possess the physical cards to play. However, the prototype does not adhere to the official trading card game rules; instead, it utilizes a custom rule set inspired by previous mobile RPGs. In this system, the physical card serves two primary purposes: selecting the player's Digimon and acting as an anchor for the 3D-model.

The game consists of three distinct scenes :

- Main Menu
- Digimon Selection Screen
- Battle Scene

The gameplay mechanics resemble a mobile Turn-Based RPG. Key resources include *Health Points* (HP) and *Attack Points* (AP), which must be strategically managed during each round.

III. REQUIREMENTS

The development of Digi-Battle was executed using the Unity Game-Engine , with scripts written in C# [3]. 3D-models were sourced from *The Models Resource*, specifically assets from the *Digimon New Century* game [2]. Note that these assets are not officially endorsed for use. To operate



Fig. 1. Digimon Card of Victory Greymon used as an image target.

the game, users require an Android device running Android Version 10 or higher, along with at least two Digimon cards (either digital images or physical prints).

IV. POSSIBLE FEATURES

As this project was designed primarily for demonstration, certain features lack the depth of a full commercial release. Future development could incorporate the following functionalities :

- **Player vs Player (PvP):**
 - Local PvP allowing two players to fight on a single device.
 - Networked PvP allowing interactions across two connected devices.
- **Buffs/Debuffs:** The current system does not yet utilize status effect mechanics.

- **Card Collection:** A feature to track the user's inventory of cards.
- **Multiple Digimon:** Expanding combat from 1v1 to team-based battles.
- **Extended Combat:** Implementing multi-round matches rather than single-round fights.
- **Official Rule Integration:** Adopting the actual Digimon Card Game ruleset.

The list above represents only a fraction of the possible enhancements available for future iterations.

V. GAMEPLAY

Upon initializing the game, the player enters the Main Menu, offering options to fight against an AI or another player. While the `Player vs Player` mode is visible, it is not currently implemented. Users may also exit the application via the `Quit` button.

Once a game mode is selected, the player proceeds to the Digimon Selection Screen. Here, the player presents the physical card of their chosen Digimon to the camera. Upon successful detection, a button appears to initiate the AI's Digimon selection. Once the second card is recognized, the game transitions to the Battle Screen.

A. Battle Mechanics

Each combatant begins with full *HP* and 4 *Attack Points* (AP). Attacks cost between 1 and 4 AP, and players recharge 1 AP per round. If a player fails to manage AP, their turn may be skipped due to insufficient points for an attack. The first player to deplete their opponent's health wins the battle. Following the match, a dialog prompts the player to either replay or return to the main menu.

VI. IMPLEMENTATION

A. Screens and Scenes

1) *Main Menu:* This scene manages the application entry point. It initiates the `GameManager`, a singleton class responsible for maintaining the selected game mode, the Digimon database, the selected characters, and the card references.

- **Player vs Player:** Displayed but not implemented (see Section IV).
- **Player vs CPU:** Transitions to the selection screen.
- **Quit:** Terminates the application.

2) *Digimon Selection:* This screen activates the Digimon selection system. The script first scans to verify, if the player has selected a Digimon. If not, the detected card and Digimon are assigned to the player in the `GameManager`. Tracking pauses, and the *Start AI Selection* button activates. Upon clicking, tracking resumes for the AI's card. Once complete, the `BattleScene` loads.

3) *Battle:* The battle interface relies on the `BattleHud` class to manage UI updates. The HUD displays Digimon names, HP, and AP. Attack buttons remain deactivated until the battle officially commences. The `StartBattle` coroutine triggers only when both physical cards are detected in the frame.

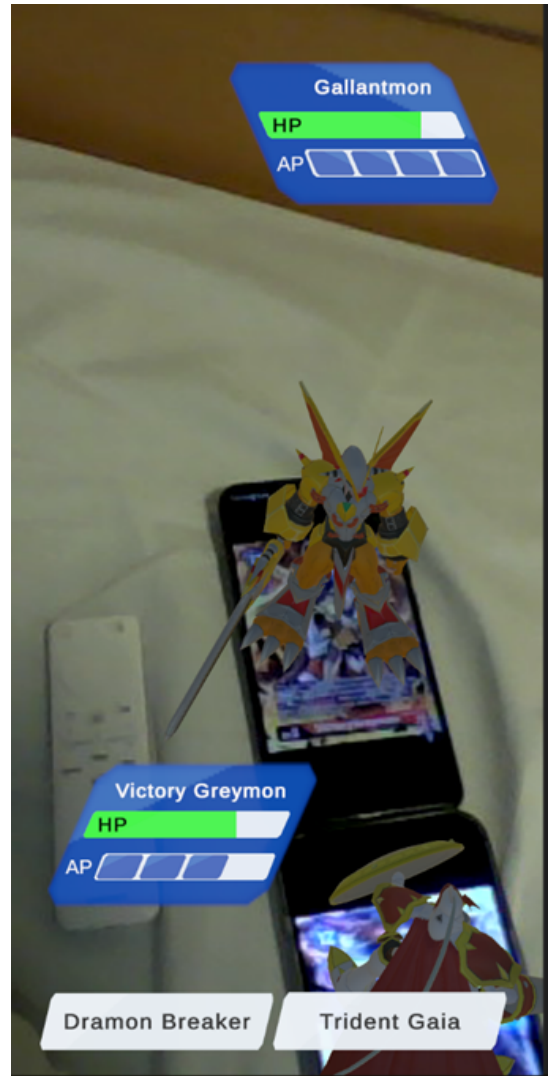


Fig. 2. An ongoing battle between the Player and the AI.

B. Digimon Selection System

This system utilizes Vuforia for image recognition [1]. Currently, it identifies preregistered image targets within the scene rather than dynamically generating them. When Vuforia detects a card, it queries the card by name and the Digimon by ID. Upon success, the `OnDigimonDetection` callback is executed.

C. Battle System

The `BattleSystem` class encapsulates the core combat logic. It provides attachable callbacks for various game states. Crucially, the logic runs within an enumerator (Coroutine), allowing asynchronous actions—such as animations—to be written in a readable, synchronous style.

VII. CODE IMPLEMENTATION SAMPLES

The following code snippets demonstrate the core architecture described in the previous section.

A. GameManager Singleton

The GameManager ensures state persistence across scenes using the Singleton pattern.

```
1 public class GameManager : MonoBehaviour {
2     public static GameManager _instance = null!;
3
4     void Awake() {
5         // If the instance reference has not been set,
6         // yet,
7         if (_instance) {
8             Destroy(gameObject);
9         } else {
10             _instance = this;
11             _instance.digimonDatabase.cards = Resources.
12             LoadAll<DigiCard>("DigimonCards").ToList();
13             _instance.digimonDatabase.AddDigimonRange(
14             Resources.LoadAll<DigimonBase>("DigimonBase"));
15         }
16
17         // Do not destroy this object, when we load a
18         // new scene.
19         DontDestroyOnLoad(gameObject);
20     }
21 }
```

Listing 1. GameManager Singleton Pattern

B. Battle Logic Coroutine

The battle flow utilizes Unity Coroutines to handle turn-based logic sequentially.

```
1 public IEnumerator RunBattle() {
2     // Start battle
3     playerController.Setup();
4     enemyController.Setup();
5
6     playerController.Start();
7     enemyController.Start();
8
9     yield return WaitForEnterAnimations();
10
11     // 1. Assign references to who goes first
12     bool playerStarts = UnityEngine.Random.Range(0,
13     2) == 0;
14
15     IController currentAttacker = playerStarts ?
16     playerController : enemyController;
17     IController currentDefender = playerStarts ?
18     enemyController : playerController;
19
20     while (!battleEnded) {
21         // 2. Run the generic turn logic
22         yield return ProcessTurn(currentAttacker,
23         currentDefender);
24         // 3. Swap roles for the next loop
25         (currentAttacker, currentDefender) = (
26         currentDefender, currentAttacker);
27     }
28
29     OnBattleEnd(battleResult);
30 }
```

Listing 2. Battle System Coroutine Structure

VIII. FUTURE WORK

The current iteration contains known bugs and lacks robust error handling or resource cleanup, which may result in memory leaks. Additionally, visual improvements, such as updating physical attacks to move models toward the enemy rather than playing animations in place, are planned.

IX. CONCLUSION

Augmented Reality has proven to be a compelling technology for enhancing tabletop gaming. By merging traditional card mechanics with modern interactive visuals, Digi-Battle offers a unique experience previously unavailable in the genre.

REFERENCES

- [1] PTC Inc., *Vuforia Engine*, [Online]. Available: <https://developer.vuforia.com/>.
- [2] The Models Resource, *Digimon New Century Models*, [Online]. Available: <https://www.models-resource.com/>.
- [3] Unity Technologies, *Unity Real-Time Development Platform* [Online]. Available: <https://unity.com/>.